НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

ПРОТОКОЛ

лабораторної роботи № 1

«Розгортання середовища та порівняльний аналіз мовних моделей»

з дисципліни «Обчислювальні методи»

Виконав:
студент групи АІ-242
Кушнір С.П.

Перевірив:
ас. Баранов О.М.

Одеса – 2026

Завдання 1. Вартість міркування: міркувальна й інструктивна моделі

Запити до обох моделей надсилалися через ендпойнт /api/generate. Спочатку моделям поставили просте фактичне запитання, після чого дали задачу на міркування (по 5 запусків на кожну модель). Перед основною серією вимірювань виконано один прогрівальний запит.

Таблиця 1 – Фактичне запитання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	654	10
Час, с	101,71	1,61
Токенів/с	7,62	20,17

Таблиця 2 – Задача на міркування (середнє значення за 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних відповідей із 5	5 / 5	2 / 5
eval_count	1145,6	177,2
Час, с	137,32	13,99

Типова помилка інструктивної моделі – відповідь «10 грн». Вона виникає, коли не враховується, що різниця в 100 грн розподіляється між двома доданками.

Відповіді на запитання (крок 6)

1. Куди витрачено додаткові токени міркувальної моделі?

Відповідь: Вони потрапили в поле thinking моделі qwen3:4b. Поле response коротке (5 грн), тоді як thinking містить великий текст обсягом 953–1395 токенів: модель складає рівняння, перевіряє себе та сумнівається. У llama3.2:3b поля thinking немає взагалі, тож усі токени одразу йдуть у response.

2. На яких типах задач міркувальні моделі мають перевагу?

Відповідь: Вони помітно випереджають звичайні моделі там, де потрібна багатокрокова логіка: математика, програмування, планування, аналіз. На логічній задачі qwen3:4b дала 5 правильних відповідей із 5, а інструктивна llama3.2:3b – лише 2 із 5.

3. Чим навчання міркувальної моделі відрізняється від навчання інструктивної?

Відповідь: Інструктивні моделі вчаться на парах «запитання – відповідь», і винагорода залежить від правильності фінального результату. Міркувальні додатково проходять навчання з підкріпленням, де оцінюється не лише відповідь, а й якість самого міркування: перевірки та виправлення помилок.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні. Це лише свідчить про більші обчислювальні витрати на «роздуми» заради точності. Токени, що не наближають до правильного результату, – марна витрата. Приклад: llama3.2:3b згенерувала менше токенів, але часто помилялася, а qwen3:4b витратила їх багато й відповіла правильно.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: Є два чинники: режим роботи й архітектура. По-перше, qwen3:4b працювала в режимі міркування й видала значно більше токенів, що потребує більше часу. По-друге, навіть темп генерації в неї нижчий, що може бути пов’язано з розміром моделі та ефективністю інференсу.

Міркувальна модель виправдана там, де ціна помилки перевищує ціну часу: багатокрокові розрахунки, планування, код. У таких випадках «тихе вгадування» часто дає хибний результат – саме так вийшло в llama3.2:3b (2 із 5). Інструктивна модель кращі там, де відповідь очевидна й важлива швидкість: прості факти, короткі формати, інтерактивні чат-боти, де користувач не хоче чекати хвилини заради того самого результату. На практиці правильне питання звучить не «яка модель краща», а «який режим підходить для цього типу задач». Якби в мене був класифікатор задач, я б рутинні та фактичні запити віддавав інструктивній моделі, а складні розрахунки й агентські ланцюжки – міркувальній, свідомо погоджуючись на витрати часу.

Висновок до завдання 1

Експеримент показав, що швидкість генерації та загальний час відповіді – це різні речі, які не можна змішувати: перша залежить від обладнання й розміру моделі, а другий – від обсягу роботи, який модель сама собі визначила. На фактичному запитанні режим міркування став чистою перевитратою: результат однаковий, а витрати різняться у 63 рази. На задачі про ручку й зошит ситуація обернена: саме розписаний хід міркувань дозволив уникнути привабливої, але хибної відповіді. Практичний підсумок: тип моделі слід підбирати під тип задачі. Для агента з 8 послідовними викликами різниця становитиме 1098,6 с проти 111,9 с, і міркувальна модель стає непридатною для налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Для кожної з двох моделей (llama3.2:3b та qwen3:4b) виконано по два прогони з різними system-повідомленнями. Запит користувача в усіх прогонах однаковий: «Поясни різницю між компілятором та інтерпретатором». Перше system-повідомлення нейтральне («Ти лаконічний технічний асистент»), друге жорстке («Відповідай лише українською, максимум двома реченнями»).

Таблиця 3 – Вплив system-повідомлення на відповідь

Модель	System	Довжина, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	2537	змішана (укр., рос., англ.)	~28	ні
llama3.2:3b	українською, 2 речення	289	українська (з помилками)	2	частково
qwen3:4b	лаконічний асистент	235	англійська	2	частково

qwen3:4b	українською, 2 речення	179	українська	2	так

Таблиця 4 – Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	55	477	532
llama3.2:3b	українською, 2 речення	59	74	133
qwen3:4b	лаконічний асистент	59	2870	2929
qwen3:4b	українською, 2 речення	68	930	998

Відповіді на запитання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: Обидві вклалися у два речення, але qwen3:4b впоралася краще: чиста українська мова й точний зміст. Відповідь llama3.2:3b теж містить 2 речення, проте з грубими мовними помилками.

2. Чи траплялося, що модель відповідала не українською, попри пряму вказівку?

Відповідь: У прогонах 1 і 3 прямої вказівки щодо мови не було. Тому llama3.2:3b у прогоні 1 змішала мови (укр., рос., англ.), а qwen3:4b у прогоні 3 відповіла повністю англійською. Отже, без явної вказівки модель за замовчуванням обирає англійську.

3. Як змінилася prompt_tokens при зміні system-повідомлення і чому?

Відповідь: Вона зросла, оскільки нове system-повідомлення довше: кожен токен інструкції додається до контексту.

4. Чому completion_tokens у qwen3:4b значно більше, ніж у llama3.2:3b, хоча видима відповідь має схожу довжину?

Відповідь: qwen3:4b є міркувальною моделлю: вона формує внутрішню чернетку, яка враховується в completion_tokens, але користувачеві не показується. llama3.2:3b – інструктивна й одразу видає остаточну відповідь.

Крок 7. Запитання до чат-бота

Дані з таблиць було надіслано чат-боту, після чого поставлено п’ять запитань.

1. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь: Моделі навчалися на різних даних і за різними стратегіями вирівнювання. У llama3.2:3b слабший instruction-following через малий розмір та обмежений україномовний корпус, тому вона зривається на домінантні мови. qwen3:4b тримає інструкції краще завдяки навчанню міркуванню, проте без вказівки мови переходить на англійську.

2. Чим system-повідомлення відрізняється від user-повідомлення з погляду моделі й чи справді має вищий пріоритет?

Відповідь: Технічно нічим: це просто текст у контексті. Утім, під час навчання модель бачила, що system зазвичай містить правила взаємодії, тому надає йому більшу вагу. Це статистична схильність, а не жорстке правило, і конкретніша та пізніша user-інструкція може її переважити.

3. Чому міркувальна модель витрачає більше вихідних токенів, навіть якщо видима відповідь однакової довжини?

Відповідь: Перед остаточною відповіддю вона генерує проміжні кроки міркування (thinking). У response вони не потрапляють, але входять у completion_tokens. Це ціна підвищеної якості на складних задачах, а на простих запитаннях – суцільні накладні витрати.

4. Що буде, якщо в system і user передати суперечливі інструкції?

Відповідь: Найімовірніше, модель виконає user-інструкцію, якщо вона конкретніша й розташована пізніше в контексті. Проте поведінка недетермінована: інколи модель видає щось середнє або ігнорує обидві.

5. Чому малі моделі гірше дотримуються мовних інструкцій, ніж великі?

Відповідь: У навчальному корпусі малих моделей української мало, а російської й англійської – багато. Тому під час генерації складних конструкцій модель перемикається на домінантну мову. Великі моделі мають більше параметрів і краще узагальнюють мовні інструкції.

System-повідомлення – найдешевший важіль керування моделлю: не потрібні ні донавчання, ні зміни коду. Проте його надійність обмежена. Прогони показали: інструкція щодо довжини виконується стабільно, а щодо мови – ні. llama3.2:3b навіть із прямою вказівкою писати українською видала суржик, а qwen3:4b без вказівки перейшла на англійську.

Висновок до завдання 2

Експеримент засвідчив, що system-повідомлення – найдешевший спосіб керування поведінкою моделі: достатньо одного рядка тексту, без донавчання та змін у коді. Водночас надійність цього способу залежить від моделі. Обмеження довжини виконували обидві моделі, а мовне – ні: llama3.2:3b за прямої вказівки все одно видала суржик із русизмами, а qwen3:4b без вказівки перейшла на англійську. Нейтральні формулювання на кшталт «лаконічний асистент» не спрацювали взагалі – обидві моделі їх проігнорували. Практичний підсумок: system-повідомлення необхідне, але недостатнє. Якщо модель його ігнорує, інструкцію варто продублювати в user-повідомленні, застосувати програмні обмеження або взяти більшу модель. Для україномовних задач qwen3:4b показала себе значно краще за llama3.2:3b: лише вона дала чисту українську без русизмів і калькувань. Ціною стали у 6–12 разів більші completion_tokens через режим міркування, що для коротких відповідей є суттєвою перевитратою.

Завдання 3. Локальна й хмарна модель: єдиний інтерфейс

Запит виконано до двох моделей через єдиний OpenAI-сумісний інтерфейс: локальної llama3.2:3b і хмарної openai/gpt-oss-120b. Ключ доступу зберігається у файлі .env, який не потрапляє до Git завдяки .gitignore. Запит користувача однаковий: «Поясни різницю між компілятором та інтерпретатором». Для кожного прогону зафіксовано час відповіді, видимий текст, мову, наявність фактичних помилок та показники response.usage.

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	openai/gpt-oss-120b
Час відповіді, с	23,80	0,92
completion_tokens	153	238
Довжина відповіді, символів	518	581
Мова відповіді	суржик (укр. + рос.)	українська
Фактичних помилок	4	0

Відповіді на запитання (крок 5)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: У логіці – жодного. У словнику PROVIDERS змінюються три значення: base_url, api_key і model (для локальної моделі api_key – формальний рядок "ollama", для хмарної – ключ зі змінної середовища GROQ_API_KEY). Сам виклик для обох моделей однаковий.

2. Яка відповідь повніша й точніша? Наведіть приклад.

Відповідь: Значно точніша відповідь хмарної openai/gpt-oss-120b. Вона правильно пояснює, що компілятор одноразово перетворює код на машинний і створює окремий виконуваний файл, а інтерпретатор читає код построчно під час виконання. Локальна llama3.2:3b вигадує неіснуючі терміни на кшталт «запам’ятовувальна система», які не мають сенсу.

3. Чи були в локальної моделі фактичні помилки або проблеми з українською?

Відповідь: Так, і чимало. По-перше, суржик із русизмами; по-друге, вигадані терміни.

4. Чому хмарна модель відповіла швидше, хоча запит іде через інтернет?

Відповідь: Groq використовує спеціалізоване обладнання LPU, оптимізоване саме для інференсу мовних моделей. Швидкість генерації сягає ~1000 токенів/с, що на порядок перевищує CPU-інференс локальної моделі. Затримка мережі на цьому тлі мізерна.

Крок 6. Запитання до чат-бота

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: OpenAI першою запровадила де-факто стандарт API для LLM, і на час появи конкурентів цей формат уже був добре знайомий розробникам. Провайдерам

вигідніше підтримати сумісність, ніж змушувати клієнтів переписувати код. Наслідок: той самий код працює з Ollama, Groq і Google, змінюються лише адреса та ключ.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір впливає на якість порівняно з іншими чинниками?

Відповідь: Розмір важливий, але не єдиний чинник. Суттєву роль відіграють якість навчальних даних, стратегія вирівнювання, архітектура та тренування на конкретних мовах. Експеримент показав: 3B-модель провалює українську не стільки через розмір, скільки через нестачу україномовних даних у корпусі.

3. Чим пояснюється, що хмарна модель відповіла швидше попри мережеву затримку?

Відповідь: Groq застосовує спеціалізовані LPU-чипи, оптимізовані під інференс LLM. Вони дають ~1000 токенів/с, тоді як CPU-інференс – близько 5–10 токенів/с. Затримка мережі (50–200 мс) на цьому тлі незначна.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Усі надіслані дані потрапляють на чужий сервер і можуть зберігатися для навчання чи аудиту. У комерційних проєктах застосовують корпоративні контракти з гарантією «no training on your data», локальні моделі для чутливих даних, маскування або анонімізацію PII перед відправленням і self-hosted рішення для критичних систем.

5. Що таке rate limit у безкоштовних тарифах і як розробники його обходять?

Відповідь: Rate limit – обмеження кількості запитів або токенів за одиницю часу. Його обходять через черги та експоненційний backoff у разі помилки 429, ротацію між кількома провайдерами, кешування однотипних запитів і платні тарифи для продакшену.

Локальну модель варто обирати, коли дані не можна передавати назовні, потрібна робота без інтернету або йдеться про навчання й прототипування, де якість не критична: вона безкоштовна, приватна й не залежить від провайдера. Хмарна модель підходить для продакшену, де важливі якість, швидкість і підтримка української: вона в
десятки разів швидша, пише чистою мовою й не вигадує термінів, але залежить від мережі, має rate limits, а всі дані надходять на чужий сервер.

Висновок до завдання 3

Експеримент показав, що завдяки OpenAI-сумісному інтерфейсу перемкнутися між локальною та хмарною моделлю можна, змінивши лише три значення в записі провайдера (base_url, api_key і model), а код виклику лишається без змін. Різниця в якості разюча: хмарна модель на 120B параметрів дала точну відповідь чистою українською за 0,92 с, а локальна 3B – суржик із вигаданими термінами за 23,80 с. За швидкістю також виграє хмарна: LPU-обладнання Groq з надлишком компенсує мережеву затримку. Локальну модель доцільно обирати, коли дані мають лишатися всередині, потрібна робота без інтернету або достатньо прототипу з невисокими

вимогами до якості. Хмарну – для продакшену з високими вимогами до якості, швидкості та української мови, з огляду на залежність від мережі, rate limits і передавання даних стороннім сервером.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Швидкість виміряно для двох провайдерів – локальної llama3.2:3b та хмарної openai/gpt-oss-120b – на трьох промптах різної довжини. Для кожного промпту виконано прогрів і три повтори, а результатом узято медіану, щоб прибрати вплив холодного старту та випадкових викидів. Окремо визначено холодний і теплий старт локальної моделі.

Таблиця 7 – Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2,082	2,392	12,6
локальна	llama3.2:3b	середній	2,094	5,066	64,7
локальна	llama3.2:3b	довгий	2,110	18,098	103,9
хмарна	openai/gpt-oss-120b	короткий	0,483	0,515	66,1
хмарна	openai/gpt-oss-120b	середній	0,316	0,566	639,1
хмарна	openai/gpt-oss-120b	довгий	0,436	2,620	1194,0

Таблиця 8 – Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	16,20	2,70
Всього, с	16,83	3,00

Крок 5. Стовпчикова діаграма

Нижче наведено діаграму швидкості генерації (символів за секунду) для всіх шести комбінацій «модель × промпт».

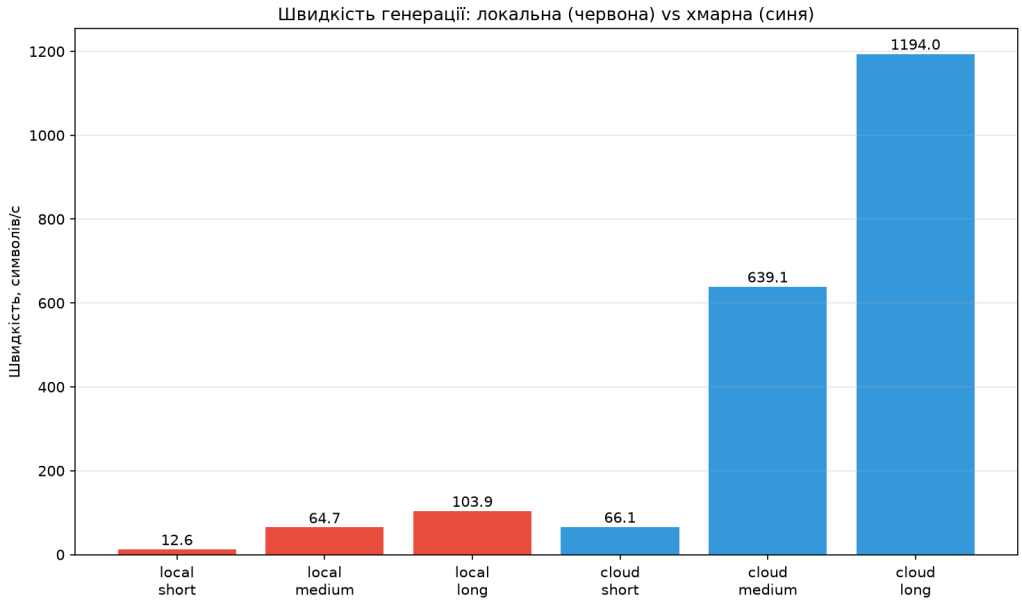


Рисунок 1 – Швидкість генерації: локальна та хмарна моделі

Крок 6. Відповіді на запитання

1. У якої моделі кращий TTFT і чому?

Відповідь: У хмарної. Groq використовує LPU-чипи з наднизькою латентністю, і мережева затримка виявляється меншою за час, який локальна CPU-модель витрачає на токенізацію та обробку промпта.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»?

Відповідь: Так, хмарна модель виграла і за TTFT, і за швидкістю генерації. Теоретично ці показники можуть розходитися: TTFT залежить від обробки промпта й мережі, а швидкість генерації – від обчислювальної потужності. У моїх вимірюваннях Groq виграв за обома складниками.

3. Як змінилося співвідношення локальної та хмарної моделі від короткого промпту до довгого?

Відповідь: На короткому промпті різниця найменша (≈5 разів за швидкістю генерації), бо значну частку часу займає TTFT: 2,08 із 2,39 с у локальної й 0,48 із 0,52 с у хмарної. Із зростанням промпта різниця збільшується до ≈10 разів на середньому й ≈11,5 разу на довгому, бо домінує генерація, де відмінність у швидкості вирішальна.

4. Наскільки різняться три повтори одного виміру? Чи вистачило б одного запуску?

Відповідь: Три повтори були стабільними: медіана близька до кожного зі значень. Проте одного запуску недостатньо: модель недетермінована, а перший запуск може бути спотворений холодним стартом. Медіана трьох запусків стійкіша до викидів.

5. Чому холодний старт повільніший за теплий?

Відповідь: Холодний старт передбачає завантаження моделі з диска в RAM (2 ГБ для llama3.2:3b), а це 13,5 с чистої затримки. За теплого старту модель уже в пам’яті, тож лишаються тільки токенізація й обробка промпта.

Крок 7. Запитання до чат-бота

1. Що фізично відбувається під час TTFT і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: Під час TTFT модель токенізує промпт, пропускає його крізь усі шари й формує перший токен. Це принципово інша фаза: prefill обробляє весь вхідний контекст одразу, а генерація видає токени по одному. TTFT визначальний для UX, а швидкість генерації – для довгих відповідей.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: У нашому випадку вона виграла за обома показниками, але теоретично TTFT залежить від мережевої затримки та черги на сервері, а швидкість генерації – від обчислювальної потужності. Якщо сервер перевантажений, TTFT може погіршитися, проте сама генерація однаково буде швидшою за локальну.

3. Чому перший запуск локальної моделі повільніший і що таке холодний старт?

Відповідь: Холодний старт – це завантаження ваг моделі з диска в оперативну пам’ять. Для llama3.2:3b це 2 ГБ, які потрібно прочитати й розмістити в RAM. Після цього модель залишається в пам’яті, і наступні запуски йдуть за теплим сценарієм.

4. Чому під час вимірювання продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Модель недетермінована, а час генерації залежить від довжини відповіді, яка щоразу різна. Медіана стійкіша до викидів, ніж середнє, і відображає типову продуктивність. Окремий запуск може випадково потрапити на холодний старт або аномально довгу відповідь.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді тієї ж довжини?

Відповідь: Prefill обробляє всі токени промпта паралельно через матричні множення, а генерація відбувається послідовно, по токену за крок. Тому опрацювати 100 токенів промпта швидше, ніж згенерувати 100 токенів відповіді, навіть за подібної обчислювальної складності.

Для інтерактивного чат-бота найважливіший TTFT: саме він визначає, чи відчує користувач «зависання». Якщо TTFT < 1 с, інтерфейс здається миттєвим. Загальний час другорядний: коротка відповідь догенерується, поки користувач читає перші слова. Groq із TTFT 0,3–0,5 с ідеально підходить для чат-ботів. Для фонового оброблення великих обсягів тексту TTFT не критичний, адже ніхто не чекає в реальному часі, і тут важливіші пропускна здатність та вартість. Локальна модель може бути вигіднішою за великих обсягів і потреби в приватності, попри повільність. Хмарна – коли потрібні швидкість і якість.

Висновок до завдання 4

Експеримент показав, що затримка відповіді складається з двох різних величин – TTFT і швидкості генерації, – які можуть суперечити одна одній. Усупереч очікуванням, хмарна модель виграла за обома: TTFT 0,32–0,48 с проти 2,08–2,11 с у локальної (у 4–7 разів), а швидкість генерації – 66,1–1194 симв./с проти 12,6–103,9 (у

5–11 разів). Мережева затримка виявилася меншою за час, який локальна CPU-модель витрачає на обробку промпта. Найвиразніше різниця проявилася на довгому промпті: 18,10 с у локальної проти 2,62 с у хмарної, тобто майже у 7 разів. Окремо виміряно холодний і теплий старт: TTFT холодного – 16,20 с, теплого – 2,70 с, а різниця в 13,5 с – це чистий час завантаження моделі в RAM. Практичний підсумок: для інтерактивних застосунків із довгими відповідями хмарна модель майже безальтернативна, а локальна годиться для коротких запитів, де TTFT близько 2 с прийнятний і важлива приватність. Для агентів із багатьма послідовними викликами різниця ще драматичніша: 8 викликів × 18 с ≈ 2,4 хвилини у локальної проти ≈21 с у хмарної.

Завдання 5. Оцінювання якості моделей за рубрикою

Виконано прогони трьох моделей – локальної інструктивної llama3.2:3b, локальної міркувальної qwen3:4b та хмарної openai/gpt-oss-120b – на фіксованому наборі з шести завдань. Оцінювання проводилося за трибальною шкалою (0 – не впоралася, 1 – частково, 2 – виконано повністю) згідно з критеріями таблиці 10 методички. Правильну відповідь на завдання 1 (рік відкриття пеніциліну) перевірено за авторитетними джерелами [1, 2], а завдання 2 – об’єктивним тестом через json.loads().

Таблиця 13 – Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	1	2	2
2. Формат	1	1	2
3. Стиснення	0	1	2
4. Код	1	1	1
5. Логіка	0	2	2
6. Українська	0	2	2
Разом із 12	3	9	11

Таблиця 14 – Розбіжність оцінок між двома прогонами

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	так	ні	ні
2. Формат	так	так	так
3. Стиснення	ні	так	ні
4. Код	ні	так (таймаут)	ні
5. Логіка	так	ні	ні
6. Українська	так	ні	ні

Крок 6. Відповіді на запитання

1. Яка модель набрала найбільше балів і на яких завданнях основна різниця?

Відповідь: Найбільше балів набрала хмарна модель (11/12), далі qwen3:4b (9/12), найменше – llama3.2:3b (3/12). Головна різниця виникла на завданнях, що потребують точних фактів, міркування або якісної мови.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це каже про надійність моделі?

Відповідь: Розбіжності: llama3.2:3b – завдання 1, 2, 5, 6; qwen3:4b – 2, 3, 4; хмарна – лише 2. Це свідчить про нестабільність llama3.2:3b: на той самий запит вона дає різні відповіді, тому ненадійна. qwen3:4b стабільніша, а хмарна – найстабільніша.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так, усі три. У всіх шести відповідях (по два прогони на модель) JSON подано без markdown-огорожі та без пояснень, і кожна відповідь парситься через json.loads() без жодних правок (скрипт check_json.py). Розбіжності були в іншому – у змісті й структурі: llama3.2:3b і qwen3:4b повернули один об’єкт, але зі змістом, що не відповідав очікуваному (безглузді значення в llama3.2:3b, інше відкриття в qwen3:4b), а хмарна модель – масив із кількох об’єктів замість одного. Отже, вимогу «без markdown» виконали всі моделі; проблеми стосувалися точності змісту та відповідності очікуваній структурі.

4. Чи були проблеми з українською: русизми, кальки, перехід на іншу мову?

Відповідь: Так: у llama3.2:3b – постійно, у qwen3:4b – один перехід на англійську в завданні 3 (прогін 2). У хмарної проблем немає, мова чиста.

Крок 7. Приклад провалу

Обраний випадок: llama3.2:3b, завдання 5 (логіка), прогін 1.

Повний текст запиту:

40 пацієнтів, 18 з температурою, 15 з кашлем, 7 з обома. У скількох немає симптомів? Покажи хід розв’язання.

Повний текст відповіді (без скорочень):

«Давайте використовуватимо принцип "всі пацієнти" або "всі пацієнти є" щоб спростувати кількість пацієнтів, які не мають симптомів.

1. Patients з температурою - 18

2. Patients з кашлем - 15

3. Patients з обома - 7

Побачивши, що п'ять пацієнтів з обома мають симптоми, ми можемо припустинити, що п'ять пацієнтів з обома не мають симптомів.

4. Patients з обома без симптомів - 7

Тепер, якщо ми зібратимо всі пацієнтів, які не мають симптомів, то ми отримаємо:

5. Patients без симптомів - 7 (з обома) + 7 (всі інші пацієнти, які не мають жодного симптому) = 7 + 33 = 40

Всего у нас 40 пацієнтів, 7 з обома не мають симптомів, а 33 з інших пацієнтів не мають жодного симптому.

Итак, 33 пацієнта не мають жодного симптому.»

Правильна відповідь: 14 пацієнтів не мають жодного симптому.
Пояснення причини помилки:

Модель не засвоїла формулу включень-виключень і почала міркувати навмання: спершу припустила, що 7 пацієнтів з обома симптомами симптомів не мають, потім

додала 7 + 33 = 40 й оголосила, що 33 пацієнти без симптомів. Це типова галюцинація в багатокроковому міркуванні: у моделі немає внутрішнього механізму перевірки, вона лише генерує правдоподібний текст. Крім того, суржик («Patients», «Всего», «Итак», «припустинити») свідчить, що модель «думає» російською, а «відповідає» українською.

Крок 8. Запитання до чат-бота

1. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: Автоматичні метрики, LLM-as-a-Judge, людське оцінювання за шкалою Лайкерта, бенчмарки та A/B-тестування в продакшені. Рубрика – найпростіший і найпрозоріший метод для невеликих наборів завдань.

2. Чому моделі часто ігнорують пряму вказівку не використовувати markdown при генерації JSON?

Відповідь: Модель навчалася на мільйонах прикладів, де JSON загорнутий у markdown-блоки. Ця закономірність настільки сильна, що перемагає пряму вказівку: модель «знає краще», як має виглядати оформлений JSON. Це фундаментальна проблема – промпт є ненадійним способом керувати форматом. Примітка: у нашому експерименті цього не сталося – усі три моделі виконали вимогу й видали JSON без markdown-огорожі (див. відповідь на запитання 3 кроку 6).

3. Чому малі моделі гірше пишуть українською, ніж англійською?

Відповідь: В навчальному корпусі української на порядки менше, ніж англійської, а російської – більше, ніж української. Модель зривається на домінантну або змішану мову. Малі моделі особливо вразливі, бо мають менше параметрів для запам’ятовування рідкісних мовних шаблонів.

4. Чому та сама модель на однаковий запит може давати відповіді різної якості?

Відповідь: Модель недетермінована: на кожному кроці вона семплує токен із розподілу ймовірностей, а не бере найімовірніший. Навіть за temperature = 0 можливі відмінності через особливості обчислень із плаваючою комою. Тому оцінка за одним прогоном ненадійна – потрібно щонайменше два.

5. Що таке галюцинація і чому модель радше вигадає факт, ніж визнає незнання?

Відповідь: Галюцинація – це генерація правдоподібного, але вигаданого тексту. Модель не має механізму «я не знаю» й завжди видає найімовірніший токен. Якщо факту немає в навчальних даних, статистично правдоподібним виявляється вигаданий варіант. Визнання незнання потребувало б окремого навчання.

Оцінювання за рубрикою виявилося надійним: три моделі отримали чітко розмежовані бали (3, 9 і 11 із 12), і ці числа можна перевірити та відтворити. Єдина складність виникла із завданням 2: буквальний критерій таблиці 10 («json.loads() парситься – 2 бали») формально дає всім трьом моделям по 2. Однак зміст JSON у локальних моделей – галюцинація («палітка на Меркурія» у llama3.2:3b, «electromagnetic induction» замість пеніциліну в qwen3:4b), тому я оцінював і формат, і зміст: локальні отримали 1 бал, хмарна – 2. Це доводить, що малий розмір сам по собі

не вирок, але для багатокрокових задач міркувальна модель обов’язкова. Чисту українську для текстового завдання дали обидві моделі – і qwen3:4b, і хмарна. Я обрав би qwen3:4b, якщо пріоритет – приватність, безкоштовність і робота без інтернету: вона пише чистою українською, стабільно розв’язує логіку й не галюцинує факти. Хмарну я обрав би, якщо пріоритет – максимальна якість і швидкість: лише вона дала правильний JSON у завданні 2 й відповідає в десятки разів швидше. llama3.2:3b непридатна для таких задач за будь-яких пріоритетів: пише суржиком, галюцинує факти й дає різні результати між прогонами.

Висновок до завдання 5

Експеримент показав, що оцінювання за рубрикою дає чіткий, відтворюваний профіль моделі: хмарна – 11 із 12, qwen3:4b – 9 із 12, llama3.2:3b – 3 із 12. Основна різниця виникла на завданнях із точними фактами, багатокроковим міркуванням і якісною мовою. llama3.2:3b провалила всі три: вигадувала роки, не розв’язала логічну задачу й писала суржиком. qwen3:4b бездоганно впоралася з логікою та мовою, але в завданні з JSON дала зміст, що не відповідав очікуваному, і лише частково впоралася зі стисненням (одна з відповідей – англійською). Хмарна – єдина, що дала правильний факт у JSON, проте й вона порушила структуру, повернувши масив замість одного об’єкта. Практичний підсумок: llama3.2:3b не можна застосовувати до задач, де потрібні факти, логіка чи українська мова; qwen3:4b – оптимальний вибір для локальної роботи з українським текстом (хоч і дуже повільний для CPU); хмарна – для задач, де потрібні максимальна якість і швидкість. Головний висновок для наступних лабораторних: розмір моделі не вирішує все, бо qwen3:4b утричі краща за llama3.2:3b завдяки навчанню міркуванню, а хмарна випередила її лише на 2 бали: різниця виникла лише на завданнях 2 і 3 (зміст JSON та мова стиснення), тоді як логіку й українську qwen3:4b виконала так само добре. Для агента, який викликає інструменти через структуровані дані, найкритичніше завдання 2. Вимогу «без markdown» виконали всі три моделі, проте жодна не повернула очікуваний один об’єкт із правильним змістом: локальні моделі видали невідповідний зміст, а хмарна – масив замість об’єкта. Отже, «чистий» JSON ще не гарантує відповідності очікуваній структурі, тому в продакшені потрібна програмна валідація (json.loads() і перевірка полів), а не покладання на текст інструкції.

Завдання 6. Керування випадковістю: temperature і top_p

Серію експериментів проведено з локальною моделлю llama3.2:3b через Ollama. Для творчого завдання (назва платформи взаємного навчання студентів) і фактичного запитання (рік відкриття пеніциліну Флемінгом) виконано по 5 запусків за різних значень temperature і top_p. Ключова метрика для творчого завдання – кількість унікальних відповідей із п’яти, для фактичного – кількість правильних. Окремо перевірено відтворюваність: 10 запусків фактичного запитання за temperature = 0.0. Правильну відповідь перевірено за авторитетними джерелами [1, 2]: пеніцилін відкрито 1928 року Олександром Флемінгом.

Таблиця 16 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	2	5
temperature	0.7	5	–
temperature	1.5	5	2
top_p	0.1	1	–
top_p	0.5	5	–
top_p	1.0	5	–

Таблиця 17 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Крок 7. Відповіді на запитання

1. Яку temperature поставити агенту, що має повертати JSON, і чому?

Відповідь: Для структурованих даних потрібна максимальна передбачуваність: той самий запит має давати той самий JSON. Експеримент це підтвердив – за temperature = 0.0 фактична відповідь була однаковою всі 10 разів.

2. А генератору варіантів заголовків?

Відповідь: Для творчих задач потрібна різноманітність. За temperature = 0.0 модель видає той самий список, за 0.7 – 5 унікальних відповідей, за 1.5 – теж 5, але серед них трапляються безглузді.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь: temperature змінює форму розподілу – згладжує або загострює ймовірності всіх токенів. top_p відсікає «хвіст» – залишає лише токени, що разом набирають p % ймовірності. В експерименті top_p = 0.1 дала 1 унікальну відповідь, а temperature = 0.0 – 2 унікальні, тож top_p сильніше обмежує різноманітність.

4. Чи означає temperature = 0 повну відтворюваність? Що показав експеримент із 10 запусків?

Відповідь: В експерименті – так: 1 унікальна відповідь із 10. Проте методичка попереджає, що гарантії немає, бо обчислення з плаваючою комою недетерміновані, а на GPU додається паралелізація. У моєму випадку модель працювала переважно на CPU, тому результат стабільний.

5. Наведіть найбезглуздішу відповідь за temperature = 1.5 і поясніть, чому вона виникла.

Відповідь: temperature = 1.5, унікальних 5 із 5:

• *Обіємо запгоронувати такі плани з відсотної процентки:*

*1. **Сімesteроносентрально**: Студенті на 100% взяли участь в онліній строчкі мадбйтере.*

*2. **Троесентрально-експериментальні**: Studenti на 66,7% взяли участь у онліній строчкі мдбйтері. Цимі обйтімі сантайся вшаноwапмйа екперменмвмей.*

*3. **Дуесentоносентрально**: Studenti делять сеймс у online навчичальї.*

Це суміш кількох мов і вигаданих слів. Вона з'явилася, бо за temperature = 1.5 розподіл сплющується: малоймовірні токени отримують шанс, і модель обирає випадкові комбінації символів.

Крок 8. Запитання до чат-бота

1. Що саме математично робить temperature з розподілом ймовірностей наступного токена?

Відповідь: Temperature ділить логіти на значення T. За T < 1 розподіл загострюється (найімовірніший токен домінує), за T > 1 – сплющується, за T = 1 залишається вихідним розподілом моделі.

2. Чим top_p відрізняється від temperature, якщо обидва впливають на різноманітність?

Відповідь: top_p сортує токени за ймовірністю й лишає тільки ті, що разом набирають p; решту відкидає до семплінгу. Temperature змінює самі ймовірності, не відкидаючи жодного токена. Тому top_p обмежує простір вибору, а temperature – форму розподілу.

3. Чи має сенс змінювати обидва параметри одночасно, чи краще налаштовувати один?

Відповідь: Зазвичай налаштовують щось одне: або temperature, або top_p. Одночасна зміна ускладнює розуміння, що саме вплинуло на результат. Типова практика: temperature – для творчих задач, top_p – для контролю «хвоста».

4. Чому навіть за temperature = 0 модель не гарантує абсолютно однакової відповіді?

Відповідь: Обчислення з плаваючою комою недетерміновані: порядок операцій, паралелізація на GPU та округлення можуть давати трохи різні логіти. В експерименті на CPU було 10/10 однакових відповідей, але на GPU результат міг би бути іншим.

5. Які значення temperature використовують для добування структурованих даних, а які – для творчого тексту?

Відповідь: Для структурованих даних – 0.0–0.2, для творчого тексту (назви, слогани, історії) – 0.8–1.2, для чат-ботів загального призначення – 0.7, для коду – 0.0–0.3.

Для наступних лабораторних, де модель має повертати структуровані дані, я обиратиму temperature = 0.0: експеримент підтвердив, що це забезпечує ідеальну передбачуваність. За temperature = 1.5 модель видала 3 з 5 неправильних років, тож випадковість тут працює проти нас. Типові налаштування (0.7–1.0) непридатні для структурованих даних: модель може вигадати зайве поле або зламати формат, тому temperature = 0 слід задавати явно. Цікаво, що за temperature = 0.0 творче завдання дало не 1, а 2 унікальні відповіді – той самий список платформ, але з різними вступними фразами. Отже, підрахунок через set – груба метрика: вона ловить різницю

у формулюваннях, а не в змісті. А top_p = 0.1 дала лише 1 унікальну відповідь із 5 – сильніше обмеження різноманітності, ніж temperature = 0.0. Це підтверджує, що параметри діють різними механізмами: temperature змінює форму розподілу, а top_p відсікає хвіст.

Висновок до завдання 6

Експеримент показав, що temperature і top_p регулюють випадковість, а не «розумність». За temperature = 0.0 модель дає передбачувані відповіді: на фактичне запитання – 1928 п’ять разів із п’яти, а в тесті відтворюваності – 10/10 однакових. За temperature = 1.5 модель вигадує: 3 з 5 років неправильні, а творчі відповіді перетворюються на безглуздий набір слів. top_p діє інакше: за top_p = 0.1 – 1 унікальна відповідь із 5, за top_p = 1.0 – 5 унікальних. Тобто top_p обмежує простір вибору, а temperature – форму розподілу. Практичний підсумок: для структурованих даних – temperature = 0.0, для творчих задач – 0.8–1.2, для чат-ботів – 0.7. Типові налаштування (0.7–1.0) не підходять для агентів: вони додають випадковість, що ламає формат.

Таблиця 18 – Підсумкове порівняння моделей

Критерій	**Локальна мала (3B)**	**Локальна більша (4B/8B)**	**Хмарна**
Швидкість генерації	12,6–103,9 симв./с	~15–25 симв./с	66,1–1194 симв./с
TTFT	2,08–2,11 с	1–3 хв (thinking)	0,32–0,48 с
Якість, балів із 12	3	9	11
Стабільність формату (завдання 2)	нестабільна (галюцинації)	нестабільна (не той факт)	відносно стабільна
Потрібні ресурси	2 ГБ RAM, CPU+GPU	3,2 ГБ RAM, CPU+GPU	GPU/CPU/RAM на сервері провайдера
Вартість	безкоштовно	безкоштовно	безкоштовно (з лімітами)
Куди йдуть дані	нікуди (локально)	нікуди (локально)	на сервер Groq
Працює без інтернету	так	так	ні
Коли обирати	прототипи, де якість не критична	локальна робота з укр. текстом, важлива приватність	максимальна якість і швидкість

Загальні висновки

1) Локальна й хмарна моделі працюють через єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах.
2) Затримка відповіді складається з двох незалежних показників – TTFT і швидкості генерації; твердження «ця модель швидша» без їх розділення некоректне.
3) Режим міркування – інструмент із чіткою сферою застосування: він окупається на багатокроковому виведенні та є чистою перевитратою на фактичних запитаннях і переказі.
4) Промпт і system-повідомлення – ненадійний засіб керування поведінкою моделі: llama3.2:3b проігнорувала вимогу лаконічності й навіть за прямої вказівки писати

українською видала суржик, а в завданні з JSON хмарна модель повернула масив замість одного об’єкта. Водночас пряму вимогу «без markdown» виконали всі три моделі. Надійність забезпечує програмна валідація, а не формулювання.
5) Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій – щонайменше двох прогонів.

Список використаних джерел

1) Ernst B. Chain – Facts. Nobel Prize in Physiology or Medicine 1945 [Електронний ресурс] // NobelPrize.org. – Режим доступу: https://www.nobelprize.org/prizes/medicine/1945/chain/facts/ (дата звернення: 12.11.2026).
2) Alexander Fleming [Електронний ресурс] // Encyclopaedia Britannica. – Режим доступу: https://www.britannica.com/biography/Alexander-Fleming (дата звернення: 12.11.2026).